

Advanced Java MCQ 2

1.

```
public class MyServlet extends HttpServlet {  
    public void init() {  
        System.out.println("Servlet Initialized");  
    }  
}
```

What is wrong with the above code?

- A) Nothing, it will run as expected
- B) `init()` should return a `String`
- C) `init()` must call `super.init()` if overridden
- D) `init()` should take `HttpServletRequest` as argument

Answer: C

When overriding `init()`, it's best practice to call `super.init()` to ensure proper initialization.

2.

```
protected void doGet(HttpServletRequest req, HttpServletResponse res) throws  
IOException {  
    PrintWriter out = res.getWriter();  
    out.print("Hello");  
    out.close();  
    out.print("World");  
}
```

What will be the result?

- A) Displays "HelloWorld"
- B) Displays "Hello" only
- C) Compilation error
- D) Runtime exception

Answer: B

After closing the `PrintWriter`, additional writes are ignored.

3.

```
HttpSession session = request.getSession(false);
```

What does this code do?

- A) Always creates a new session
- B) Returns null if no session exists
- C) Returns an existing session or creates a new one
- D) Throws exception if session doesn't exist

Answer: B

`getSession(false)` returns the current session or null if none exists.

4.

```
<% session.setAttribute("username", "Alice"); %>
```

Where will this attribute be stored?

- A) In request scope
- B) In page scope
- C) In session scope
- D) In application scope

Answer: C

`session.setAttribute` stores the attribute in the session scope.

5.

```
public void doFilter(ServletRequest request, ServletResponse response,
    FilterChain chain)
    throws IOException, ServletException {
    System.out.println("Filter before");
    chain.doFilter(request, response);
    System.out.println("Filter after");
}
```

What is the output if a request goes through this filter?

- A) Only "Filter before"
- B) Only "Filter after"
- C) Both lines
- D) No output

Answer: C

The filter prints before and after processing the chain.

6.

```
@WebListener
public class MyListener implements ServletContextListener {
    public void contextInitialized(ServletContextEvent sce) {
        System.out.println("Context initialized");
    }
}
```

When is this listener triggered?

- A) On every request
- B) On session creation
- C) On application startup
- D) On JSP load

Answer: C

`ServletContextListener` triggers during application startup and shutdown.

7.

```
<%! int counter = 0; %>
<%= ++counter %>
```

What is the output after 3 consecutive page loads?

- A) 1 2 3
- B) 1 1 1
- C) Compilation error
- D) Unpredictable

Answer: A

`<%! %>` declares class-level variables that persist across requests.

8.

```
<c:if test="${5 > 3}">
    Greater
</c:if>
```

What is the output?

- A) Nothing
- B) Error
- C) Greater
- D) 5 > 3

Answer: C

The condition evaluates to true, so content is printed.

9.

```
<c:forEach var="i" begin="1" end="3">
    ${i}
</c:forEach>
```

Output?

- A) 1 2 3
- B) 0 1 2
- C) 1 2
- D) Nothing

Answer: A

Iterates from 1 to 3.

10.

```
<c:set var="name" value="Tom" />
${name}
```

Output?

- A) \${name}
- B) Tom
- C) name
- D) Error

Answer: B

c:set stores a value in page scope, accessible via EL.

11.

```
<fmt:formatNumber value="12345.678" type="currency"/>
```

Assuming US locale, output?

- A) \$12,345.68
- B) 12345.68
- C) 12,345.678
- D) \$12,345.678

Answer: A

`fmt:formatNumber` formats number as currency.

12.

```
getServletContext().setAttribute("global", "value");
```

This value is accessible in:

- A) Single request only
- B) Single session only
- C) Whole application
- D) Not accessible anywhere

Answer: C

`ServletContext` attributes are shared across the entire app.

13.

```
<filter-mapping>
  <filter-name>MyFilter</filter-name>
  <url-pattern>/*</url-pattern>
</filter-mapping>
```

What does this configuration mean?

- A) Filter applies to one URL
- B) Filter never runs
- C) Filter applies to all URLs
- D) Error in config

Answer: C

`/*` means all URL patterns.

14.

Which listener detects session creation?

- A) `ServletContextListener`
- B) `HttpSessionListener`
- C) `ServletRequestListener`
- D) `HttpSessionAttributeListener`

Answer: B

HttpSessionListener is used for session lifecycle events.

15.

```
<%@ page session="false" %>
<%= session.getId() %>
```

Result?

- A) Session ID
- B) Compilation error
- C) Runtime error
- D) null

Answer: C

Session is disabled; accessing it throws exception.

16.

Which is **not** an implicit JSP object?

- A) request
- B) session
- C) context
- D) out

Answer: C

context is not an implicit object. application is.

17.

```
<fmt:formatDate value="${now}" pattern="dd-MM-yyyy"/>
```

What must be true for this to work?

- A) now is a String
- B) now is a Date
- C) now is int
- D) Doesn't matter

Answer: B

`fmt:formatDate` requires a `java.util.Date` object.

18.

```
protected void doPost(...) {  
    doGet(request, response);  
}
```

Is this valid?

- A) Yes, calls doGet on POST
- B) Invalid
- C) Causes infinite loop
- D) Not recommended

Answer: A

Valid but only recommended if GET and POST share logic.

19.

```
@WebServlet("/hello")
```

Which URL calls the servlet?

- A) /hello
- B) /Hello
- C) /hi
- D) /HelloServlet

Answer: A

URL pattern is case-sensitive.

20.

In Servlet, what's the difference?

```
request.getRequestDispatcher("next.jsp").forward(request, response);  
response.sendRedirect("next.jsp");
```

- A) Both are the same
- B) Forward happens on server, redirect on client

- C) Redirect is faster
- D) Forward changes URL in browser

Answer: B

Explanation: Forward is internal; redirect involves client.

21.

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/fake" %>
```

What will happen?

- A) Nothing
- B) Tag works
- C) Error loading taglib
- D) Compiles, fails at runtime

Answer: C

Invalid URI causes taglib load failure.

22.

```
public void init(ServletConfig config) {  
    String param = config.getInitParameter("configParam");  
}
```

To ensure `configParam` works, where must it be defined?

- A) web.xml
- B) ServletContext
- C) URL
- D) Java class

Answer: A

: `init-param` values are defined in `web.xml`.

23.

Given two filters:

- AuthFilter
- LogFilter

If mapped as:

```
<filter-mapping>
  <filter-name>AuthFilter</filter-name>
  <url-pattern>/*</url-pattern>
</filter-mapping>
<filter-mapping>
  <filter-name>LogFilter</filter-name>
  <url-pattern>/*</url-pattern>
</filter-mapping>
```

What is the execution order?

- A) Random
- B) Log → Auth
- C) Auth → Log
- D) Parallel

Answer: C

Filters execute in the order defined in `web.xml`.

24.

```
<c:catch var="error">
  ${1/0}
</c:catch>
<c:out value="${error}" />
```

Output?

- A) 0
- B) Error message
- C) 1
- D) 0.0

Answer: B

Division by zero caught in `c:catch`, stored in `error`.

25.

```
session.invalidate();
session.setAttribute("x", "y");
```

What happens?

- A) Attribute set
- B) Error

- C) Attribute ignored
- D) Runtime exception

Answer: D

Setting attribute after invalidation causes `IllegalStateException`.

26.

```
<%= application.getServerInfo() %>
```

What does this return?

- A) Servlet name
- B) JSP file name
- C) Web server information
- D) Request URI

Answer: C

`application` is an implicit object of type `ServletContext`, and `getServerInfo()` returns server details.

27.

```
<%= config.getInitParameter("siteName") %>
```

Where is "siteName" defined?

- A) Inside web.xml under `<servlet>`
- B) Inside JSP
- C) In request scope
- D) In context-param

Answer: A

`config` accesses servlet-specific init parameters defined in web.xml under `<init-param>`.

28.

```
<% request.setAttribute("color", "blue"); %>  
<%= request.getAttribute("color") %>
```

What is the output?

- A) null
- B) blue

- C) color
- D) Error

Answer: B

Attribute set in request scope is accessible during the request's lifetime.

29.

```
<%! int counter = 0; %>
<%= ++counter %>
```

Which scripting element is `<%! ... %>`?

- A) Expression
- B) Scriptlet
- C) Declaration
- D) Comment

Answer: C

`<%! %>` is used to declare variables and methods at the class level in JSP.

30. Implicit Object - `out`

```
<% out.println("Hello"); %>
```

What type is `out`?

- A) `PrintWriter`
- B) `JspWriter`
- C) `OutputStream`
- D) `ServletOutputStream`

Answer: B

`out` is an implicit object of type `JspWriter`, used for outputting content in JSP.

31.

Which of the following is **not** a common use of filters?

- A) Authentication
- B) Logging
- C) Request modification
- D) Compiling Java code

Answer: D

Filters are used for preprocessing/postprocessing requests, not for compiling code.

32.

```
<%  
    application.setAttribute("framework", "Spring");  
%>
```

How long is this attribute available?

- A) One request
- B) One session
- C) Until the application is restarted
- D) Until the page is reloaded

Answer: C

Application scope lasts for the lifetime of the web application.

33.

```
<c:set var="theme" value="dark" scope="session" />
```

Where is `theme` stored?

- A) page
- B) request
- C) session
- D) application

Answer: C

The `scope` attribute defines where the variable lives; here it's `session`.

34.

What is the correct JSP comment syntax that won't be sent to the browser?

- A) `<!-- comment -->`
- B) `<%-- comment --%>`
- C) `<jsp:comment>`
- D) `// comment`

Answer: B

`<%-- --%>` is a JSP comment that is not included in the HTML response.

35.

Which listener is used to track session attribute changes?

- A) `HttpSessionListener`
- B) `HttpSessionAttributeListener`
- C) `ServletContextListener`
- D) `HttpServletRequestListener`

Answer: B

This listener is used to detect when attributes are added/removed/replaced in a session.

36.

Which scope is **not** accessible using `pageContext`?

- A) `page`
- B) `request`
- C) `session`
- D) `servlet`

Answer: D

`pageContext` can access `page`, `request`, `session`, and `application` scopes—not `servlets` directly.

37.

```
<% request.setAttribute("city", "Delhi"); %>
${city}
```

What is the output?

- A) city
- B) Delhi
- C) null
- D) Compilation error

Answer: B

EL can access request-scoped attributes directly by name.

38.

How do you access a context param (like `companyName`) in JSP?

- A) `${applicationScope.companyName}`
- B) `${initParam.companyName}`
- C) `${config.companyName}`
- D) `${requestScope.companyName}`

Answer: B

EL accesses context init parameters using `initParam`.

39.

Which listener is triggered **first** during web application startup?

- A) `ServletContextListener`
- B) `HttpSessionListener`
- C) `HttpServletRequestListener`
- D) `ServletRequestAttributeListener`

Answer: A

`ServletContextListener` runs when the application context is initialized, before sessions or requests.

40.

Which is **true**?

- A) `ServletConfig` is shared across servlets
- B) `ServletContext` is per servlet
- C) `ServletContext` is shared application-wide
- D) `ServletConfig` accesses context-param

Answer: C

`ServletContext` is shared across the entire web application.